

Introducing NVIDIA's Compute Unified Device Architecture (CUDA)

This article, the first in a series, introduces readers to the NVIDIA CUDA architecture, as good programming requires a decent amount of knowledge about the architecture.



Jack Dongarra, professor at the University of Tennessee and author of Linpack has said, "Graphics Processing Units have evolved to the point where many real-world applications are easily implemented on them, and run significantly faster than on multi-core systems. Future computing architectures will be hybrid systems with parallel-core GPUs working in tandem with multi-core CPUs."

Project managers often instruct developers to improve their algorithm so that their compute efficiency of their application increases. We all know parallel processing is faster, but there was always a doubt whether it would be worth the effort and time—but not any more! Graphics Processing Units (GPUs) have evolved to flexible and powerful processors, which are now programmable using high-level languages supporting 32-bit and 64-bit floating-point precision and do not require programming in assembly. They offer a lot of computational power, and this is the primary reason that developers today are focussing on getting the maximum benefit of this extreme scalability.

In the last few years, mass marketing of multi-core

GPUs has brought terascale computing power to laptops and petascale computing power to clusters. A CPU + GPU is a powerful combination, because CPUs consist of a few cores optimised for serial processing, while GPUs consist of thousands of smaller, more efficient cores designed for parallel performance. Serial portions of the code run on the CPU, while parallel portions run on the GPU.

The Compute Unified Device Architecture (CUDA) is a parallel programming architecture developed by NVIDIA. CUDA is the computing engine in NVIDIA GPUs that gives developers access to the virtual instruction set and memory of the parallel computational elements in the CUDA GPUs, through variants of industry-standard programming languages. Exploiting data parallelism on the GPU has become significantly easier with newer programming models like OpenACC, which provides developers with simple compiler directives to run their applications in parallel on the GPU.

Recently, at the 19th IEEE HiPC conference held in Pune, I met several delegates from academia and industry who wanted to make use of this extreme computing power,